

به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر

درس پردازش زبان طبیعی

تمرین ششم

استاد درس: دکتر هشام فیلی

سرپرست دستیاران آموزشی: میلاد محمدی و امیرحسین صفدریان

طراحان تمرین: مهدی صبور و حسین لشگری

دی ۱۴۰۴

فهرست

۲	مقدمه
۳	Coding Agent (نمره ۱۰۰)
۳	توسعه ابزارها (نمره ۲۵)
۴	طراحی گراف و مدیریت State (نمره ۲۵)
۴	توسعه Human In The Loop (نمره ۱۵)
۴	مدیریت حافظه (نمره ۱۰)
۵	توسعه و اتصال به CLI (نمره ۱۵)
۵	ارزیابی عامل (نمره ۱۰)
۵	موارد امتیازی (نمره ۱۵)
۶	راه اندازی و کتابخانه های مورد استفاده
۶	راه اندازی پروژه
۷	کتابخانه های مورد استفاده
۹	پروژه های ارزیابی
۹	پردازشگر قدیمی CSV (Legacy CSV Processor)
۱۰	سیستم جستجوی TinyRAG
۱۱	مدل زبانی Bigram
۱۲	نکات تحویل (حتما مطالعه شود).

در سال‌های اخیر، Large Language Model ها از بات‌های گفتگوی ساده به موتورهای استدلال قدرتمند تبدیل شده‌اند که قادر به اجرای Workflow های پیچیده هستند. در حالی که چت‌بات‌های اولیه مانند ChatGPT می‌توانستند در صورت درخواست، قطعه کدهایی تولید کنند، اما فاقد Agency (عاملیت) برای تعامل با دنیای واقعی بودند. آن‌ها نمی‌توانستند در فایل‌های سیستم جستجو کنند، کامپایلری را برای بررسی خطاها اجرا کنند، یا یک Test Suite را برای تایید منطق خود اجرا نمایند. این محدودیت به این معنا بود که کاربر انسانی همیشه باید به عنوان Integrator عمل می‌کرد. یعنی کپی کردن کد، اجرای آن، بازگرداندن خطاها به مدل و تکرار دستی این حلقه.

در این تمرین شما به طراحی و پیاده‌سازی یک Coding Agent خواهید پرداخت. به طور مشخص این عامل قادر است در یک Codebase جستجو کند، خطاها را دیباگ کند، قابلیت‌های جدیدی اضافه نماید و تست‌ها و کدها را اجرا نماید. در حقیقت این عامل به مجموعه ابزاری که توسط شما توسعه پیدا خواهد کرد دسترسی خواهد داشت تا قابلیت‌های فوق را انجام دهد. برای پیاده‌سازی عامل و ابزارهای مورد نیاز از فریم‌ورک LangChain و LangGraph استفاده خواهید کرد. همچنین از مدل‌های زبانی بزرگ بدون نیاز به فاین‌تیون کردن و از طریق API به عنوان هسته پردازشی زبان استفاده خواهید کرد. مدل پیشنهادی ما gpt-5-mini است ولی از مدل‌های دیگر نیز می‌توانید استفاده کنید.

راه ارتباط و چت کاربر با عامل از طریق یک Command Line Interface (CLI) خواهد بود. کدهای مربوط به یک نمونه CLI آماده، در پوشه coding-agent/src موجود است که می‌توانید آن را استفاده کرده و گسترش دهید یا با استفاده از ابزارهای دیگر CLI مورد نظر خود را از ابتدا توسعه دهید.

اهداف تمرین:

- ❖ کار با فریم‌ورک LangGraph و طراحی گراف‌های Stateful
- ❖ کار با ابزارهای ساخت Command Line و مدیریت جلسه
- ❖ طراحی ابزارهای مناسب برای تجهیز عامل به قابلیت‌های خواندن و نوشتن و جستجو فایل‌ها و اجرای دستورات.
- ❖ پیاده‌سازی Human In The Loop با هدف جلوگیری از تغییر فایل‌ها و اجرای دستورات بدون اجازه کاربر.
- ❖ ارزیابی عامل در پروژه‌های نرم‌افزاری که دارای مشکل و باگ هستند.

Coding Agent (نمره ۱۰۰)

همانطور که در مقدمه ذکر شد، Coding Agent به عنوان یک دستیار برنامه‌نویسی به کاربر در رفع مشکلات و ایجاد قابلیت‌های جدید کمک کند. با مشخص کردن مسیر ذخیره پروژه در سیستم برای عامل و تعامل با آن توسط CLI و بیان درخواست‌ها به زبان طبیعی، عامل از طریق ابزارهایی که در اختیار دارد شروع به بررسی فایل‌ها و خواندن آن‌ها و همچنین اجرا و تست آن‌ها می‌کند تا با تغییر کدها خواسته کاربر را رفع کند.

در ادامه اجزای عامل که باید پیاده‌سازی شود تشریح می‌شود:

توسعه ابزارها (۲۵ نمره)

برای آنکه عامل بتواند با محیط خود یعنی پروژه کاربر ارتباط برقرار کند، نیاز به پیاده‌سازی توابعی (ابزارهایی) است که به کمک LangGraph پیاده‌سازی خواهند شد. مجموعه ابزارهای مورد نیاز شامل موارد زیر است: (محدود به این موارد نیستند و در صورت نیاز می‌توانید ابزارهای دیگری نیز پیاده‌سازی کنید).

- ❖ **خواندن فایل:** این ابزار امکان خواندن محتویات یک فایل متنی مشخص را به عامل می‌دهد.
- ❖ **ایجاد فایل:** این ابزار امکان ایجاد یک فایل جدید را به عامل می‌دهد.
- ❖ **بازنویسی فایل:** این ابزار امکان بازنویسی یک فایل موجود را به عامل می‌دهد.
- ❖ **لیست فایل‌ها:** این ابزار امکان مشاهده محتویات (فایل‌ها و پوشه‌ها) یک پوشه مشخص را به عامل می‌دهد. وجود این ابزار برای درک ساختار پروژه برای عامل ضروری است.
- ❖ **جستجو فایل:** مسیر فایل‌های موجود با یک نام مشخص شده را به عامل برمی‌گرداند.
- ❖ **اجرای دستورات شل (Shell):** این ابزار امکان اجرای کدها و دستورات مورد نیاز را به عامل می‌دهد. نتیجه و یا خطای اجرا به عامل برگردانده می‌شود.

طراحی گراف و مدیریت State (۲۵ نمره)

گراف حالات عامل، مهمترین نقش در نحوه پردازش اطلاعات و استفاده بهینه از ابزارها را دارد. هر چه این گراف مناسب‌تر باشد، عامل با مصرف توکن ورودی و خروجی کمتر به جواب درست می‌رسد. بنابراین استفاده از یک معماری مناسب نقش تعیین‌کننده‌ای در عملکرد عامل دارد. شما باید با استفاده از LangGraph و انتخاب یک معماری مناسب مانند ReAct, Plan and Execute, Reflexion به پیاده‌سازی گره‌ها و یال‌های گراف بپردازید.

توسعه Human In The Loop (۱۵ نمره)

یک عامل خودکار که دسترسی نوشتن و اجرای دستورات را دارد، می‌تواند خطرناک باشد. برای جلوگیری از تغییرات ناخواسته یا اجرای دستورات مخرب، شما باید مکانیزم نظارتی انسانی را پیاده‌سازی کنید.

به این منظور ابتدا ابزارهای حساس را شناسایی کنید. در هنگام درخواست عامل برای استفاده از این ابزارها اجرای گراف متوقف شده و با نشان دادن نام ابزار و Argument‌های آن به کاربر در CLI، از کاربر تاییدیه (بله/خیر) بگیرید. در صورت تایید، ابزار اجرا شده و خروجی آن به عامل برمی‌گردد و در صورت عدم تایید بدون اجرای ابزار باید یک پیغام خطا در قالب خروجی ابزار به عامل برگردانده شود تا عامل متوجه شود اجازه آن کار را ندارد و راه دیگری برای حل مسئله بیابد.

مدیریت حافظه (۱۰ نمره)

یکی از موارد ضروری در چت با عامل ایجاد حافظه برای آن است. عامل باید بتواند در یک گفتگو پیام‌های قبلی را ببیند (با ذکر آن‌ها در Context عامل) تا در صورت نیاز از اطلاعات آن‌ها استفاده کند و پیوستگی پاسخ‌ها را رعایت کند. برای پیاده‌سازی این مورد می‌توانید از LangGraph استفاده کنید.

توسعه و اتصال به CLI (۱۵ نمره)

برای ارتباط عامل با کاربر باید از Command Line Interface کمک بگیرید. پس از طراحی گراف و ابزارها باید آن را به نحو مناسب به CLI متصل کنید. یک نمونه آماده از CLI در پوشه `coding-agent/src` موجود است که در بخش بعدی با جزئیات توضیح داده می‌شود. در صورت نیاز می‌توانید از آن استفاده کرده و گسترش دهید یا از ابتدا طراحی کنید. توجه داشته باشید تمام تعامل کاربر با عامل از طریق CLI صورت می‌گیرد، بنابراین پاسخ عامل و وضعیت میانی آن مثل اجرای ابزار را به شکل مناسب به کاربر نمایش دهید.

ارزیابی عامل (۱۰ نمره)

برای ارزیابی عملکرد عامل سه نمونه پروژه کوچک نرم‌افزاری طراحی شده است که در پوشه `coding-agent/test_projects` قرار گرفته‌اند. این پروژه‌ها دارای باگ‌هایی هستند که باعث عملکرد اشتباه آن‌ها می‌شود. عامل را باید بر روی هر یک از این پروژه‌ها جداگانه اجرا کرده و از او به زبان طبیعی بخواهید باگ‌ها را پیدا و رفع کند. برای هر پروژه یک فایل `test` طراحی شده که به عامل کمک می‌کند تا زمان پاس شدن تست‌ها به رفع مشکلات ادامه دهد. جزئیات مربوط به این پروژه‌ها در بخش‌های بعدی توضیح داده می‌شود.

موارد امتیازی (۱۵ نمره)

- ❖ **ثبت مصرف (Usage Tracker) (۵ نمره):** سیستمی را پیاده‌سازی و به عامل اضافه کنید تا میزان مصرف توکن ورودی و خروجی در هر نوبت و کل گفتگو را محاسبه و به کاربر نمایش دهد.
- ❖ **مدیریت هوشمند Context (۵ نمره):** یکی از مشکلات عوامل کدنویس، پر شدن `Context Window` با محتوای فایل‌های طولانی و خطاها است. مکانیزمی طراحی کنید که پس از اتمام کار با یک فایل یا تشخیص اینکه محتوای فایل دیگر مورد نیاز نیست، آن را از حافظه عامل حذف یا خلاصه کند تا در هزینه‌ها صرفه‌جویی شود.
- ❖ **ذخیره و بازیابی نشست‌ها (۵ نمره):** قابلیت به عامل و CLI اضافه کنید که به کاربر اجازه دهد وضعیت فعلی گفتگو را ذخیره کند (مثلا در یک فایل JSON) و بعدا بتواند ادامه کار را از همان نقطه پیگیری نماید. این قابلیت باید هم تاریخچه پیام‌ها و هم وضعیت داخلی گراف را بازیابی کند.

راهاندازی و کتابخانه‌های مورد استفاده

راهاندازی پروژه

برای راهاندازی پروژه که در پوشه coding-agent موجود است، ترجیحا از پایتون نسخه ۳.۱۱ و بالاتر استفاده کنید. در ابتدا یک Virtual Environment بسازید. سپس برای نصب پروژه و کتابخانه‌های وابسته از دستور زیر استفاده کنید:

```
pip install -e .
```

دستور فوق فایل pyproject.toml را خوانده و پکیج coding-agent روی محیط مجازی نصب می‌شود. پس از نصب پروژه می‌توانید با دستور coding-agent، عامل را اجرا کنید. در صورت نصب صحیح خروجی زیر را مشاهده می‌کنید:

```
Usage: coding-agent [OPTIONS] COMMAND [ARGS]...

Professional CLI-based coding agent with LangGraph.

Run 'coding-agent chat' to start an interactive session.

Options:
  --version Show the version and exit.
  --help Show this message and exit.

Commands:
  chat Start an interactive multi-turn coding session.
  version Display version information.
```

برای شروع گفتگو با عامل، از دستور chat استفاده کنید. شما باید مسیر پروژه هدف (پروژه‌ای که عامل قرار است روی آن کار کند) را با -p مشخص کنید:

```
coding-agent chat -p ./my-project
```

پس از اجرای این دستور، وارد محیط تعاملی Interactive CLI می‌شوید که با کتابخانه Rich طراحی شده است. در اینجا می‌توانید به زبان طبیعی از عامل درخواست کنید:

```
Coding Agent
----- Welcome -----
Project: test projects\legacy_csv_processor
Type your coding tasks or questions. Use /help for commands.

Available Commands:
----- Help -----
/help      - Show this help message
/exit      - Exit the session

Tips:
- The agent remembers your conversation history
- Recently accessed files are tracked automatically
- Use natural language to describe tasks

You: The csv processor fails on transactions data. Please fix it until pass the test file.
Agent:
----- Response -----
Your agent response
```

با مطالعه کدهای موجود در پوشه `src/coding_agent/cli` می‌توانید این محیط را گسترش داده و امکانات دیگری به رابط کاربری اضافه کنید.

برای استفاده از API Key ترجیحا آن را مشابه فایل `env.example` در یک فایل `env` تعریف کرده و به عنوان یک متغیر محیطی استفاده کنید.

کتابخانه‌های مورد استفاده

❖ **فریم‌ورک LangGraph**: این کتابخانه قدرتمند برای ساخت برنامه‌های هوشمند و چندمرحله‌ای با LLMها طراحی شده است. برخلاف زنجیره‌های خطی ساده، LangGraph به شما امکان می‌دهد منطق عامل خود را به صورت یک گراف طراحی کنید که شامل حلقه‌های تکرار است. این ویژگی برای پیاده‌سازی معماری‌هایی که نیاز به تفکر، اقدام و بازنگری دارند (مانند زمانی که عامل پس از بروز خطا باید دوباره تلاش کند) حیاتی است. در این تمرین، شما از LangGraph برای تعریف نودهای تصمیم‌گیری، نودهای ابزار و مدیریت حافظه عامل استفاده خواهید کرد.

❖ **کتابخانه Rich و اجزای رابط کاربری:** کتابخانه Rich یک ابزار قدرتمند پایتونی است که وظیفه

تبدیل ترمینال‌های سیاه و سفید قدیمی به محیط‌های تعاملی، رنگی و ساختاریافته را بر عهده دارد. در توسعه CLI این پروژه، از چندین کامپوننت کلیدی این کتابخانه استفاده شده است که هر یک در حد آشنایی در ادامه توضیح داده می‌شوند:

- **کنسول (Console):** کلاس Console جایگزین print استاندارد پایتون می‌شود. از شیء console

برای چاپ متن‌های فرمت‌دهی شده استفاده شده است. شما می‌توانید با استفاده از تگ‌هایی مانند [bold red] یا [cyan] متن‌ها را رنگ‌آمیزی و برجسته کنید تا خطاها یا پیام‌های مهم عامل بهتر دیده شوند.

- **پنل (Panel):** کلاس Panel برای کادربندی متن‌ها و ایجاد بلوک‌های بصری مجزا استفاده

می‌شود. پیام‌های خوش‌آمدگویی، راهنما و پاسخ‌های عامل درون یک Panel قرار گرفته‌اند. این کامپوننت دارای ویژگی‌هایی مانند title (برای نمایش عنوان روی کادر) و border_style (برای تغییر رنگ کادر) است که به تفکیک بصری پیام‌های کاربر از پیام‌های عامل کمک شایانی می‌کند.

- **پرامپت (Prompt):** کلاس Prompt ابزاری پیشرفته برای دریافت ورودی از کاربر است. متد

Prompt.ask علاوه بر دریافت متن، قابلیت‌هایی نظیر نمایش متن پیش‌فرض، اعتبارسنجی ورودی و رنگ‌آمیزی متن پرسش را فراهم می‌کند.

- **جدول (Table):** کلاس Table برای نمایش داده‌های ساختاریافته استفاده می‌شود. با استفاده از

این کلاس می‌توان داده‌ها را در قالب یک جدول منظم و خوانا به کاربر گزارش داد.

پروژه‌های ارزیابی

برای ارزیابی عملکرد عامل هوشمند شما، سه پروژه نرم‌افزاری معیوب طراحی شده است. هر یک از این پروژه‌ها شامل فایل‌های منبع، داده‌ها و یک مجموعه تست است که در حال حاضر با شکست مواجه می‌شود. وظیفه عامل این است که مانند یک مهندس نرم‌افزار، وارد پوشه پروژه شده، ساختار آن را درک کند، تست‌ها را اجرا کرده و با اصلاح کدها، باگ‌های موجود را برطرف نماید.

پردازشگر قدیمی CSV (Legacy CSV Processor)

این برنامه یک ابزار خط فرمان قدیمی است که وظیفه دارد فایل‌های CSV شامل تراکنش‌های مالی را پردازش کند. برنامه باید فایل را بخواند، تراکنش‌های موفق را فیلتر کند و سپس مجموع درآمد و میانگین تراکنش‌ها را محاسبه نماید. عامل باید کدهای پروژه را بررسی کند، تست‌ها را اجرا نماید و باگ‌های مربوط به انواع داده و نحوه Load کردن و حالت‌های مرزی (Edge) را رفع کند. نکته مهم: این کد اجازه استفاده از کتابخانه pandas را ندارد و پارس کردن فایل باید به صورت دستی (توسط `file_handler.py`) انجام شود.

پوشه این پروژه شامل:

- `src/file_handler.py`: وظیفه خواندن فایل متنی، جدا کردن وی‌رگول‌ها و ساخت لیست دیکشنری‌ها را دارد.
- `src/analyzer.py`: وظیفه محاسبات ریاضی را بر عهده دارد.
- `main.py`: نقطه ورود برنامه که فایل‌ها را فراخوانی می‌کند.
- `data/transactions.csv`: یک فایل استاندارد شامل ستون‌های `id,amount,status,date`.
- `data/empty.csv`: یک فایل خالی برای تست کردن پایداری برنامه.
- `tests/test_analyzer.py`: این فایل شامل ۴ تست زیر است:
 - ✓ `test_load_transactions`: بررسی صحت بارگذاری داده‌ها.
 - ✓ `test_calculate_total_revenue`: بررسی می‌کند که جمع کل درآمد دقیقاً برابر ۱۳۵۰.۵۰ باشد.
 - ✓ `test_handle_empty_file`: بررسی می‌کند که اگر فایل خالی داده شد، برنامه کرش نکند و مقدار ۰.۰ برگرداند.
 - ✓ `test_get_average_transaction`: بررسی صحت محاسبه میانگین.

سیستم جستجوی TinyRAG

این پروژه یک پیاده‌سازی مینیمال از سیستم RAG (Retrieval Augmented Generation) است که اسناد متنی را تکه‌تکه (Chunk) کرده و با استفاده از شباهت برداری فرکانس کلمات جستجو می‌کند. اما نتایج جستجو بسیار بی‌کیفیت هستند. عامل باید منطق ریاضی و پردازش متن را بررسی کرده و اشتباهات الگوریتمی را اصلاح کند.

پوشه این پروژه شامل:

- ingest/loader.py: تابعی ساده برای بارگذاری محتوای فایل‌های متنی از دیسک.
- ingest/chunker.py: مسئول تقسیم متن به قطعات کوچکتر براساس کلمات است.
- retrieval/embedding.py: مسئول تبدیل متن به بردار عددی بر اساس شمارش کلمات است.
- retrieval/search.py: مسئول محاسبه شباهت بین بردار پرس‌وجو (Query) و بردارهای اسناد است.
- database/store.py: یک پایگاه داده ساده درون‌حافظه‌ای برای نگهداری اسناد و بردارها است.
- data/knowledge_base.txt: یک فایل متنی طراحی شده برای تست که شامل جملات معنادار و جملات نویز (مانند تکرار کلمه the) است.
- tests/test_tinyrag.py: یک مجموعه تست که شامل موارد زیر است.
- ✓ *test_chunking_logic*: عملکرد مدل را در فرایند chunking در یک مورد خاص بررسی می‌کند.
- ✓ *test_search_quality*: در یک مورد خاص بررسی می‌کند که آیا جستجوی یک عبارت کلیدی، سند مرتبط را پیدا می‌کند یا سند نویز را.
- ✓ *test_similarity_score*: بررسی می‌کند که شباهت یک سند با خودش باید دقیقاً ۱.۰ باشد.

مدل زبانی Bigram

این پروژه یک مدل زبانی آماری از نوع Bigram است که تلاش می‌کند احتمال وقوع کلمه بعدی را صرفاً بر اساس کلمه قبلی محاسبه کند. از این مدل برای دو هدف استفاده می‌شود: تولید متن جدید و امتیازدهی به جملات. در حال حاضر، این مدل در مواجهه با کلمات جدید دچار خطا می‌شود، در جملات طولانی دقت ریاضی خود را از دست می‌دهد و متن‌های تولید شده توسط آن تکراری و مصنوعی هستند. عامل هوشمند باید منطق ریاضی احتمالات را اصلاح کرده و مکانیزم تولید متن را بهبود بخشد.

پوشه این پروژه شامل:

- `model/bigram.py`: کلاس اصلی `BigramModel` که وظیفه آموزش (شمردن کلمات) و محاسبه احتمالات را بر عهده دارد.
- `model/tokenizer.py`: یک ماژول کمکی ساده برای تبدیل متن به توکن‌های کوچک‌شده.
- `generation/sampler.py`: کلاس `Generator` که با استفاده از مدل آموزش‌دیده، کلمه به کلمه متن تولید می‌کند.
- `data/corpus.txt`: مجموعه‌ای از جملات ساده انگلیسی (مانند `The cat sat on the mat`) که برای آموزش مدل استفاده می‌شود.
- `tests/test_bigram.py`: یک مجموعه تست که شامل موارد زیر است.
 - ✓ `test_smoothing_unseen_bigrams`: بررسی می‌کند که اگر یک جفت کلمه (Bigram) در داده‌های آموزش وجود نداشت، احتمال آن صفر نشود.
 - ✓ `test_underflow_long_sequence`: یک دنباله طولانی از کلمات را به مدل می‌دهد تا بررسی کند آیا احتمال کل جمله به صفر میل می‌کند یا خیر.
 - ✓ `test_generation_diversity`: بررسی می‌کند که اگر ۱۰ بار از مدل بخواهیم متنی تولید کند، خروجی‌ها متفاوت باشند.

نکات تحویل (حتما مطالعه شود).

- ۱- ذخیره‌سازی logها: پس از پیاده‌سازی کامل، در طول اجرای عامل در پروژه‌های ارزیابی، گفت‌وگوهای انجام شده و وضعیت میانی آن (شامل استفاده از ابزارها و state) در یک فایل (مثلا json, txt) ذخیره شود.
- ۲- فیلم ارائه تمرین: در انتهای تمرین یک فیلم ۵ الی ۸ دقیقه‌ای از نحوه کلی پیاده‌سازی شامل معماری گراف و ابزارها و همچنین عملکرد عامل در پروژه‌های ارزیابی تهیه کنید. در این فیلم نیازی به ارائه جزئیات نیست ولی توانایی‌ها و ویژگی‌های اصلی عامل نمایش داده شود.
- ۳- کدهای شما باید قابلیت اجرا داشته باشند. اگر از کتابخانه‌ای به غیر از موارد ذکر شده استفاده کرده‌اید آن را عنوان کرده و در صورت نیاز به تنظیمات خاص آن را ذکر نمایید.
- ۴- تمامی نتایج شما شامل فایل کدهای پیاده‌سازی و پروژه‌های ارزیابی اصلاح شده توسط عامل، فیلم ارائه و logها باید در یک فایل فشرده با عنوان NLP-HW6-StudentID تحویل داده شود.
- ۵- این تمرین امکان ارسال با تاخیر ندارد. تمرین آخر شامل ارسال با تاخیر (با یا بدون گریس) نمی‌شود.
- ۶- توجه کنید این تمرین باید به صورت تک‌نفره انجام شود و نتایج ارائه شده باید حاصل فعالیت فرد نویسنده باشد. در صورت مشاهده تشابه به همه افراد مشارکت‌کننده نمره صفر تعلق می‌گیرد و به استاد نیز گزارش می‌گردد.
- ۷- در صورت بروز هرگونه مشکل با ایمیل‌های زیر در تماس باشید:

- lashgari.hn@gmail.com
- msabour.official@gmail.com

تاریخ آپلود تمرین	۱۴ دی ۱۴۰۴
مهلت تحویل بدون جریمه	۱۶ بهمن ۱۴۰۴
مهلت تحویل با تاخیر	ندارد!